Enabling Documentation-Aware Language Features in SourceKit-LSP Using Symbol Graphs

Abstract

Swift DocC allows developers to write documentation containing semantic references to API declarations. However, SourceKit-LSP currently provides no language features for documentation content because documentation references are not represented in the indexing infrastructure used for source code. As a result, authors often need to build documentation bundles to verify references and navigate to declarations.

This work integrates symbol graph generation into SourceKit-LSP's build and indexing workflows, providing a source of semantic information for documentation symbols. A key challenge was reconciling module-level symbol graph generation with SourceKit-LSP's existing single-file indexing architecture. To address this, symbol graph generation was integrated into both build and background indexing workflows while preserving existing indexing behavior. This paper discusses the design and implementation of this integration and how this infrastructure forms the foundation for implementing documentation language features.

1. Motivation

SourceKit-LSP relies heavily on IndexStore to provide semantic language features such as go-to-definition. While this works well for source code, documentation references embedded in DocC comments, Markdown files and tutorial files cannot be resolved through the existing indexing infrastructure because the Swift compiler ignores documentation comments during compilation and as a result documentation symbol references are never indexed. Consequently, SourceKit-LSP has no semantic information available for documentation symbols and cannot provide language features for documentation content.

The goal of this work is to make documentation symbols available to SourceKit-LSP and establish the infrastructure required for documentation-aware language features.

2. Symbol Graph Generation and Synchronization

Since documentation references are unavailable through IndexStore, symbol graphs were introduced as an alternative source of semantic information. Symbol graph generation was integrated into the build pipeline using the Swift compiler's `-emit-symbol-graph` flag together with `-experimental-skip-all-function-bodies` to reduce compilation overhead.

The primary challenge arose during background indexing. SourceKit-LSP's indexing workflow operates on individual source files, whereas symbol graph generation requires module-level compilation. Initial attempts to combine both workflows resulted in compiler invocation conflicts, including clashes between module output paths and supplementary output file maps inherited from the build system.

The final design separates these responsibilities. Existing indexing tasks continue unchanged, while symbol graph generation is performed through a dedicated module-level compiler invocation after indexing completes. Symbol graphs are generated per module and regenerated when files within the corresponding module change, ensuring documentation metadata remains synchronized with the project state.

3. Documentation Symbol Resolution

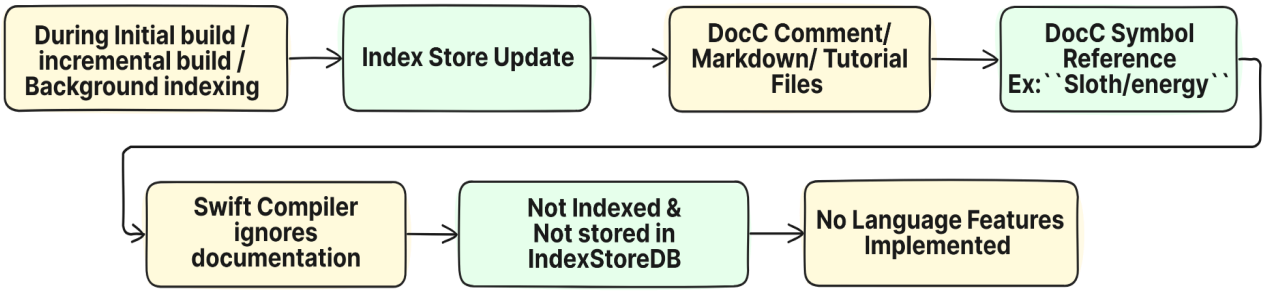
The generated symbol graphs provide the semantic information required to resolve documentation references. For Markdown and tutorial files, the Swift Markdown parser is used to construct a Markdown AST and identify the symbol reference at the user's cursor position. For Swift source files, SwiftSyntax is used to locate documentation comments, after which the comment contents are parsed using the same Markdown infrastructure. This provides a common mechanism for extracting documentation symbols across all supported documentation formats.

Once a symbol reference has been identified, the corresponding module symbol graph is queried to locate the referenced declaration. The source location recorded in the symbol graph is then returned to the editor as the definition result.

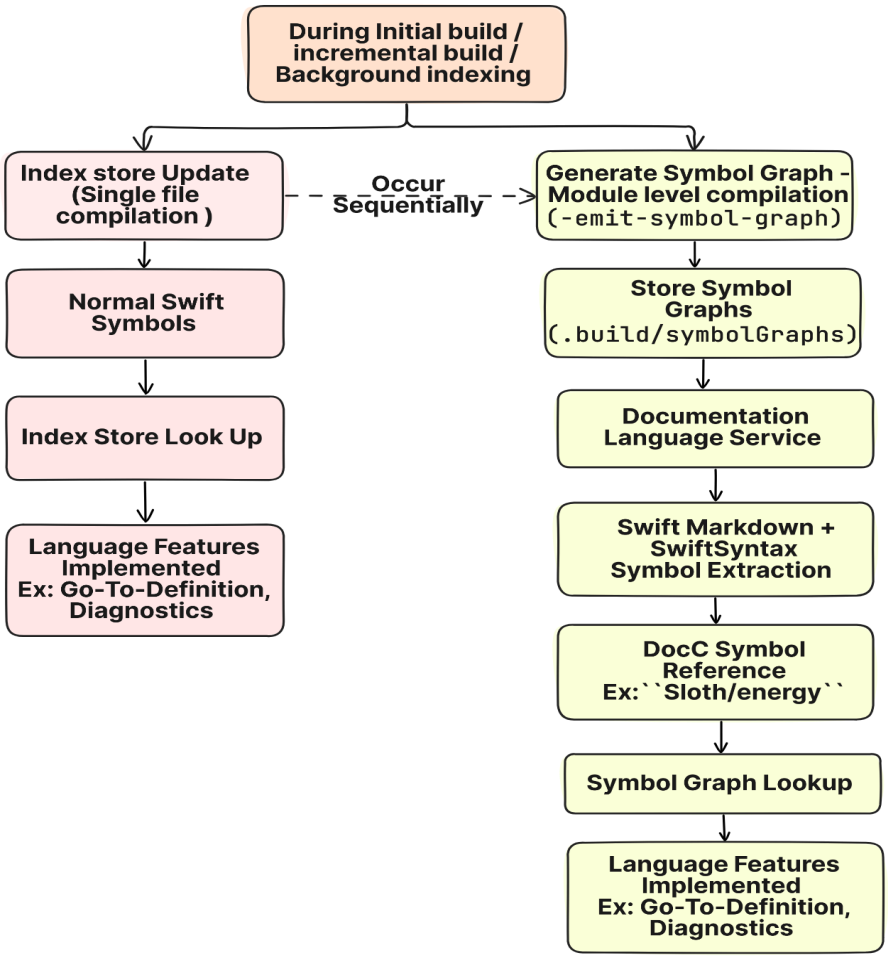
This infrastructure is currently used to implement go-to-definition support for documentation comments, Markdown files, and tutorial files within SourceKit-LSP, and can be extended to support additional documentation-aware language features

4. Architectural Overview

Before:



After:



5. Current Status and Future Work

At the time of writing, the work described in this paper is under active development and has been proposed upstream for review. The implementation includes symbol graph generation during builds and background indexing, along with documentation symbol resolution and go-to-definition support for documentation comments, Markdown files, and tutorial files. However, the changes have not yet been merged into SourceKit-LSP.

The remaining work primarily involves addressing review feedback, resolving outstanding integration issues, and refining the implementation before upstream adoption. Given the current progress, the infrastructure and language features described in this paper are expected to be completed by the time of the conference.

Beyond go-to-definition, the same infrastructure can support additional documentation language features such as diagnostics for unresolved references, hover information, code actions, interactive previews, and syntax highlighting. Longer-term features such as find references and rename support can also build upon the same symbol resolution infrastructure.

References

[1] SourceKit-LSP - https://github.com/swiftlang/sourcekit-lsp

[2] Swift Markdown - https://github.com/swiftlang/swift-markdown

[3] SwiftSyntax - https://github.com/swiftlang/swift-syntax

[4] Padmashree S S "Add go-to-definition support for DocC documentation content." SourceKit-LSP Pull Request #2698 - https://github.com/swiftlang/sourcekit-lsp/pull/2698